# PROJECT REPORT DOCUMENTATION

## MedSafe AI

AI-Driven Medical Safety Assistant

### PROJECT DETAILS

**Date:** 03 March 2026

**Team ID:** 217

**Project Name:** MedSafe AI – AI-Driven Medical Safety Assistant

**Max Marks:** 4 Marks

### PROJECT LINKS

**GitHub:** github.com/AdiPsy2175/medsafe-ai-ui

**Live Demo:** drive.google.com/file/d/1T8Bghkdftmnprq4RVilfQfepA4QljWc_/view

Developed with Python · Streamlit · Rule-based AI · Deployed on Streamlit Community Cloud

# Table of Contents


1. **Project Title** ..... 3
2. **Project Description** ..... 3
   - System Overview ..... 3
   - Problem Domain ..... 4
   - Core Technology Used ..... 4
   - Key Capabilities ..... 4
   - Target Users ..... 4
   - Real-world Relevance ..... 4
3. **Application Scenarios / Use Cases** ..... 5
4. **Technical Architecture Overview** ..... 5
5. **Prerequisites** ..... 6
   - 5.1 Software Requirements ..... 6
   - 5.2 Libraries / Frameworks / Dependencies ..... 6
   - 5.3 Hardware Requirements ..... 6
6. **Prior Knowledge Required** ..... 6
7. **Project Objectives** ..... 7
8. **System Workflow** ..... 7
9. **Milestone 1: Requirement Analysis & Design** ..... 7
   - 9.1–9.5 Problem Definition, Requirements, Design ..... 7
10. **Milestone 2: Environment Setup** ..... 8
    - 10.1–10.4 Setup, Installation, Configuration ..... 8
11. **Milestone 3: Core System Development** ..... 8
    - 11.1–11.3 Feature Implementations ..... 9
12. **Milestone 4: Integration & Optimisation** ..... 9
13. **Milestone 5: Testing & Validation** ..... 9
    - 13.1–13.4 Test Cases, Unit, Integration, UAT ..... 9
14. **Deployment** ..... 10
15. **Project Structure** ..... 10
16. **Results** ..... 10
17. **Advantages & Limitations** ..... 11
18. **Future Enhancements** ..... 11
19. **Conclusion** ..... 11
20. **Appendix** ..... 11

# 1. PROJECT TITLE

MedSafe AI – AI-Driven Medical Safety Assistant

# 2. PROJECT DESCRIPTION

## System Overview

MedSafe AI is an AI-assisted healthcare support system designed to enhance medication safety awareness and provide basic symptom guidance through an interactive web interface. The system enables users to check potential drug interactions and understand symptoms experienced after medication consumption.

The application is built using Python and Streamlit, providing a lightweight yet powerful environment for developing an intelligent medical guidance tool. The system processes user inputs and produces simplified explanations regarding medication safety.

## Problem Domain

Medication misuse and lack of awareness about drug interactions remain common problems in healthcare. Many patients consume multiple medications simultaneously without understanding potential interactions or side effects. Additionally, individuals experiencing symptoms after medication often struggle to determine whether their condition requires medical attention.

Traditional solutions such as searching online medical resources can be confusing and time-consuming due to complex medical terminology. Therefore, there is a need for a simple system that can provide quick and understandable guidance.

MedSafe AI aims to bridge this gap by providing an accessible digital assistant that helps users make safer decisions regarding medication and health awareness.

## Core Technology Used

The system uses a combination of modern web technologies and lightweight AI logic:

- Python – Core programming language used to implement application logic.
- Streamlit – Framework used to develop the interactive web interface.
- Rule-based reasoning engine – Used to evaluate medicine interactions and symptoms.
- Git & GitHub – Version control and collaborative development platform.
- Streamlit Community Cloud – Cloud hosting platform used for application deployment.

## Key Capabilities

The MedSafe AI system offers several useful features designed to improve medication safety awareness:

- Drug interaction analysis based on user-provided medicine names.
- Symptom guidance system that helps users understand potential side effects.
- Simplified explanations generated using AI-style reasoning.
- User-friendly web interface accessible from any browser.
- Educational health guidance that promotes responsible healthcare decisions.

## Target Users

The system is designed to assist a wide range of users including:

- Patients taking multiple medications
- Elderly individuals managing prescriptions
- Caregivers monitoring patient health
- Individuals experiencing symptoms after medication
- Students learning about healthcare safety

## Real-world Relevance

Medication safety is an important aspect of public health. Many avoidable health issues arise due to improper medication usage or lack of awareness about drug interactions.

MedSafe AI provides an accessible digital tool that helps users better understand medication safety risks and encourages responsible health decisions. While it does not replace professional medical consultation, it serves as an educational support tool that promotes healthcare awareness.

# 3. APPLICATION SCENARIOS / USE CASES

### Healthcare Awareness

Individuals can use the system to check whether multiple medicines can be safely taken together, particularly useful for patients managing chronic conditions who are prescribed several drugs simultaneously.

### Personal Health Monitoring

Users experiencing symptoms after taking medication can describe their symptoms and receive guidance about possible risks, helping them decide whether to seek professional medical attention.

### Educational Use

Medical or healthcare students can use the platform as a simple learning tool to understand medication safety concepts and drug interaction principles in an accessible format.

### Caregiver Assistance

Caregivers can monitor the medication safety of elderly family members and better understand possible side effects, enabling them to provide better care and flag concerns early.

# 4. TECHNICAL ARCHITECTURE OVERVIEW

The MedSafe AI system follows a modular architecture consisting of three major layers that work together to process user inputs and generate results.

## Frontend Layer

The user interface is developed using Streamlit, allowing users to interact with the system through a web browser. The interface provides input fields for entering medicine names or describing symptoms.

## Backend Layer

The backend layer processes user inputs using Python-based logic modules. These modules analyse drug interactions and symptom descriptions to generate meaningful guidance.

Main backend components include:

- Medicine interaction analysis module (interaction_logic.py)
- Symptom analysis module (symptom_logic.py)
- Explanation generation module (ai_explainer.py)

## Deployment Layer

The application is deployed using Streamlit Community Cloud, which automatically builds and hosts the application from the GitHub repository. Any push to the main branch triggers a rebuild and redeploy.

## System Request Flow

1. The user accesses the MedSafe AI web interface.
2. The user enters medicine names or describes symptoms.
3. The system receives and validates the input.
4. Backend modules process the request.
5. Interaction or symptom analysis is performed.
6. AI-style explanations are generated.
7. Results are displayed to the user through the interface.

# 5. PREREQUISITES

## 5.1 Software Requirements

To run the project successfully, the following software components are required:

- Python 3.9 or later
- Visual Studio Code (recommended IDE)
- Git for version control
- Web browser (Chrome / Edge / Firefox)
- Streamlit Community Cloud for deployment

### 5.2 Libraries / Frameworks / Dependencies

The following library is required:

- Streamlit

Install all dependencies using:

```
pip install -r requirements.txt
```

### 5.3 Hardware Requirements

The project does not require specialised hardware. A standard computer with the following specifications is sufficient:

- Minimum 4 GB RAM
- Internet connection
- Modern web browser

## 6. PRIOR KNOWLEDGE REQUIRED

Developers working on this project should have knowledge of:

- Python programming fundamentals
- Web application basics
- Streamlit framework concepts
- Git and GitHub workflow
- Basic understanding of healthcare information systems

## 7. PROJECT OBJECTIVES

### Technical Objectives

Develop an intelligent web application capable of analysing medicine interactions and providing symptom guidance using a rule-based AI reasoning engine.

### Performance Objectives

Ensure fast processing of user inputs and generate quick responses with minimal latency, relying on lightweight local logic rather than external APIs.

### Deployment Objectives

Deploy the application online using Streamlit Community Cloud for free, public accessibility without requiring server management.

### Learning Outcomes

- Understand AI-assisted healthcare applications
- Learn web development using Streamlit
- Gain experience with GitHub version control
- Understand cloud deployment of web applications

## 8. SYSTEM WORKFLOW

The system workflow describes the step-by-step execution process of the application:

8. User opens the MedSafe AI web interface.
9. User enters medicine names or symptom descriptions.
10. The application validates the input.
11. Backend modules process the data.
12. Drug interaction or symptom analysis is performed.
13. AI-style explanation is generated.
14. The result is displayed to the user.

# 9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

## 9.1 Problem Definition

There is a lack of simple tools that allow users to understand medication safety and possible drug interactions in an easy and accessible way. Existing resources are often too technical for general users.

## 9.2 Functional Requirements

- Accept medicine names as user input
- Analyse potential drug interactions
- Accept symptom descriptions
- Provide clear guidance messages

## 9.3 Non-Functional Requirements

- Simple and intuitive interface
- Fast response time
- Web-based accessibility — no installation required

## 9.4 System Design Decisions

The project uses a lightweight web application architecture to ensure easy development and quick deployment. A modular design separates the UI, logic, and explanation layers for maintainability.

## 9.5 Technology Stack Selection

- Python – for backend processing and logic implementation
- Streamlit – for rapid UI development
- GitHub – for project version control
- Streamlit Community Cloud – for free, automated cloud hosting

# 10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

## 10.1 Development Environment Setup

The development environment includes Python, Git, and Visual Studio Code installed on the local machine.

## 10.2 Dependency Installation

Required dependencies are installed using the requirements file:

```
pip install -r requirements.txt
```

## 10.3 Project Structure Creation

A structured repository was created to organise application code, documentation, and testing files. See Section 15 for the full folder structure.

## 10.4 Configuration Setup

The GitHub repository is connected to Streamlit Cloud for automated deployment. Any push to the main branch triggers an automatic rebuild and redeploy.

# 11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

## 11.1 Feature 1 – Medicine Interaction Checker

Users can input multiple medicine names, and the system analyses possible interactions between them using the rule-based interaction_logic.py module. The system returns a risk level (None / Moderate / High) and an appropriate warning message for each pair.

## 11.2 Feature 2 – Symptom Guidance Module

Users can describe symptoms they experience after medication. The symptom_logic.py module analyses keyword-based input and classifies the risk as Low, Moderate, or High, providing relevant educational guidance.

## 11.3 Feature 3 – AI-Style Explanation Generator

The ai_explainer.py module converts raw risk assessments from both modules into plain-language, easy-to-understand explanations. This ensures the output is accessible to non-medical users including elderly patients and caregivers.

# 12. MILESTONE 4: INTEGRATION & OPTIMISATION

## 12.1 Component Integration

The Streamlit interface was integrated with backend analysis modules. The app.py file coordinates all three modules and manages the user session flow.

## 12.2 Performance Optimisation

Code was optimised to reduce processing time and improve user interaction speed. Rule-based logic is executed efficiently without external API calls, ensuring instant responses.

## 12.3 Security Enhancements

Basic input validation mechanisms were implemented to handle invalid or empty inputs, preventing errors and unexpected behaviour during runtime.

## 12.4 Error Handling

The system displays clear error messages when invalid inputs are provided, guiding the user to correct their input and retry.

# 13. MILESTONE 5: TESTING & VALIDATION

## 13.1 Test Cases

| Test Case ID | Input | Expected Output | Status |
|---|---|---|---|
| TC01 | Valid medicine names | Interaction analysis displayed | ✓ Passed |
| TC02 | Empty input | Warning message displayed | ✓ Passed |
| TC03 | Symptom description | Guidance result displayed | ✓ Passed |

## 13.2 Unit Testing

Individual modules (interaction_logic.py, symptom_logic.py, ai_explainer.py) were tested independently to ensure correct processing of various user inputs and edge cases.

## 13.3 Integration Testing

Frontend and backend components were tested together to ensure smooth interaction and correct data flow from input to output.

## 13.4 User Acceptance Testing

The application was evaluated for usability, clarity of results, and overall user experience by testing with representative users from the target audience.

# 14. DEPLOYMENT

## 14.1 Deployment Architecture

The application is deployed using Streamlit Community Cloud, which provides free, automated hosting directly from a GitHub repository.

## 14.2 Hosting Platform

**Platform:** Streamlit Community Cloud

## 14.3 Deployment Steps

15. Push project code to the GitHub repository.
16. Connect the repository with Streamlit Community Cloud.
17. Streamlit Cloud automatically builds and deploys the application.
18. Access the live application at the deployment URL.

## 14.4 Live Demo

**Live Application:** drive.google.com/file/d/1T8Bghkdftmnprq4RVilfQfepA4QljWc_/view

# 15. PROJECT STRUCTURE

The following folder structure organises all project files:

```
medsafe-ai-ui/
│
├── Brainstorming & Ideation/
├── Requirement Analysis/
├── Project Planning Phase/
├── Project Design Phase/
├── Project Documentation/
│
├── app.py                  ←  Main Streamlit application
├── interaction_logic.py    ←  Medicine interaction module
├── symptom_logic.py        ←  Symptom analysis module
├── ai_explainer.py         ←  AI explanation generator
├── requirements.txt        ←  Python dependencies
├── test_cases.md           ←  Test documentation
└── README.md               ←  Project overview
```

# 16. RESULTS

The system successfully allows users to check medicine interactions and analyse symptoms through a simple web interface. The deployed application works correctly on Streamlit Community Cloud and demonstrates the feasibility of AI-assisted healthcare guidance systems.

**Outcome:** All three core features — Medicine Interaction Checker, Symptom Guidance, and AI Explanation Generator — are fully functional and deployed.

# 17. ADVANTAGES & LIMITATIONS

### Advantages

- Simple and user-friendly interface accessible from any browser
- Fast response system with no external API dependency
- Accessible 24/7 via Streamlit Community Cloud deployment
- Designed for non-medical users with plain-language explanations

### Limitations

- Uses simplified rule-based logic rather than real medical databases
- Does not replace professional medical consultation
- Limited to predefined medicine and symptom datasets

## 18. FUTURE ENHANCEMENTS

- Integration with real pharmaceutical databases (e.g. RxNorm, DrugBank)
- AI-based symptom prediction models using machine learning
- Mobile application version (iOS / Android)
- OCR-based prescription scanning using image recognition
- Integration with healthcare APIs for real-time drug data

## 19. CONCLUSION

The MedSafe AI project demonstrates how artificial intelligence and web technologies can be combined to build an educational healthcare assistance tool. The system successfully provides medication safety guidance and symptom analysis through a simple web interface.

The project highlights the potential of AI-assisted applications in improving healthcare awareness, particularly for non-medical users who need clear and accessible information about medication safety. The modular architecture, clean codebase, and cloud deployment make it a strong foundation for future enhancements.

> **Technical Achievement:** MedSafe AI successfully integrates rule-based AI reasoning with a Streamlit interface, deployed on Streamlit Community Cloud — demonstrating a complete end-to-end AI-assisted healthcare web application.

## 20. APPENDIX

### GitHub Repository

**Repository URL:** https://github.com/AdiPsy2175/medsafe-ai-ui

### Live Demo

**Deployed Application:** drive.google.com/file/d/1T8Bghkdftmnprq4RVilfQfepA4QljWc_/view

### Technology References

- Streamlit Documentation: https://docs.streamlit.io
- Python Documentation: https://docs.python.org
- GitHub: https://github.com
- Streamlit Community Cloud: https://streamlit.io/cloud